

```

import string

class Vectorizer:
    def standardize(self, text):
        text = text.lower()
        return "".join(char for char in text if char not in string.punctuation)

    def tokenize(self, text):
        text = self.standardize(text)
        return text.split()

    def make_vocabulary(self, dataset):
        self.vocabulary = {"": 0, "[UNK]": 1}
        for text in dataset:
            text = self.standardize(text)
            tokens = self.tokenize(text)
            for token in tokens:
                if token not in self.vocabulary:
                    self.vocabulary[token] = len(self.vocabulary)
        self.inverse_vocabulary = dict(
            (v, k) for k, v in self.vocabulary.items())

    def encode(self, text):
        text = self.standardize(text)
        tokens = self.tokenize(text)
        return [self.vocabulary.get(token, 1) for token in tokens]

    def decode(self, int_sequence):
        return " ".join(
            self.inverse_vocabulary.get(i, "[UNK]") for i in int_sequence)

vectorizer = Vectorizer()
dataset = [
    "I write, erase, rewrite",
    "Erase again, and then",
    "A poppy blooms.",
]
vectorizer.make_vocabulary(dataset)

```

```

test_sentence = "I write, rewrite, and still rewrite again"
encoded_sentence = vectorizer.encode(test_sentence)
print(encoded_sentence)

```

```

→ [2, 3, 5, 7, 1, 5, 6]

```

```

decoded_sentence = vectorizer.decode(encoded_sentence)
print(decoded_sentence)

```

```

→ i write rewrite and [UNK] rewrite again

```

```

from tensorflow.keras.layers import TextVectorization
text_vectorization = TextVectorization(
    output_mode="int",
)

```

```

dataset = [
    "I write, erase, rewrite",
    "Erase again, and then",
    "A poppy blooms.",
]
text_vectorization.adapt(dataset)

```

Doing text standardization and Vocab stuff

```
rm -rf aclImdb
```

```
rm -rf aclImdb_v1.tar.gz
```

```
!curl -O https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
!tar -xvf aclImdb_v1.tar.gz
```

```

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total       Spent    Left     Speed

100 80.2M  100 80.2M    0     0  37.6M      0  0:00:02  0:00:02 --:--:-- 37.6M

```

```
!rm -r aclImdb/train/unsup
```

```
!cat aclImdb/train/pos/4077_10.txt
```

```

I first saw this back in the early 90s on UK TV, i did like it then but i missed the chance to tape it, many years passed but the film a

```

```
import os, pathlib, shutil, random
```

```

base_dir = pathlib.Path("aclImdb")
val_dir = base_dir / "val"
train_dir = base_dir / "train"
for category in ("neg", "pos"):
    os.makedirs(val_dir / category)
    files = os.listdir(train_dir / category)
    random.Random(1337).shuffle(files)
    num_val_samples = int(0.2 * len(files))
    val_files = files[-num_val_samples:]
    for fname in val_files:
        shutil.move(train_dir / category / fname,
                    val_dir / category / fname)

```

```
print(int(0.2 * len(files)))
```

```
2500
```

Start coding or [generate](#) with AI.

```

-----
NameError                                Traceback (most recent call last)
<ipython-input-1-12e59ea44076> in <cell line: 0>()
----> 1 print(int(0.2 * len(files)))

NameError: name 'files' is not defined

```

```

from tensorflow import keras
batch_size = 32

```

```

train_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/train", batch_size=batch_size
)
val_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/val", batch_size=batch_size
)
test_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/test", batch_size=batch_size
)

```

```

Found 20000 files belonging to 2 classes.
Found 5000 files belonging to 2 classes.
Found 25000 files belonging to 2 classes.

```

```

for inputs, targets in train_ds:
    print("inputs.shape:", inputs.shape)
    print("inputs.dtype:", inputs.dtype)
    print("targets.shape:", targets.shape)
    print("targets.dtype:", targets.dtype)
    print("inputs[0]:", inputs[0])
    print("targets[0]:", targets[0])
    break

```

```

inputs.shape: (32,)
inputs.dtype: <dtype: 'string'>
targets.shape: (32,)
targets.dtype: <dtype: 'int32'>
inputs[0]: tf.Tensor(b'Boring, badly written Italian exploitation flick.Lots of nudity, gore and awful acting.The werewolf makeup was th
targets[0]: tf.Tensor(0, shape=(), dtype=int32)

```

```

text_vectorization = TextVectorization(
    max_tokens=20000,
    output_mode="multi_hot",
)
text_only_train_ds = train_ds.map(lambda x, y: x)
text_vectorization.adapt(text_only_train_ds)

binary_1gram_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
binary_1gram_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
binary_1gram_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)

for inputs, targets in binary_1gram_train_ds:
    print("inputs.shape:", inputs.shape)
    print("inputs.dtype:", inputs.dtype)
    print("targets.shape:", targets.shape)
    print("targets.dtype:", targets.dtype)
    print("inputs[0]:", inputs[0])
    print("targets[0]:", targets[0])
    break

inputs.shape: (32, 20000)
inputs.dtype: <dtype: 'int64'>
targets.shape: (32,)
targets.dtype: <dtype: 'int32'>
inputs[0]: tf.Tensor([1 1 1 ... 0 0 0], shape=(20000,), dtype=int64)
targets[0]: tf.Tensor(0, shape=(), dtype=int32)

from tensorflow import keras
from tensorflow.keras import layers

def get_model(max_tokens=20000, hidden_dim=16):
    inputs = keras.Input(shape=(max_tokens,))
    x = layers.Dense(hidden_dim, activation="relu")(inputs)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)
    model = keras.Model(inputs, outputs)
    model.compile(optimizer="rmsprop",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])
    return model

model = get_model()
model.summary()
callbacks = [
    keras.callbacks.ModelCheckpoint("binary_1gram.keras",
                                   save_best_only=True)
]
model.fit(binary_1gram_train_ds.cache(),
          validation_data=binary_1gram_val_ds.cache(),
          epochs=10,
          callbacks=callbacks)
model = keras.models.load_model("binary_1gram.keras")
print(f"Test acc: {model.evaluate(binary_1gram_test_ds)[1]:.3f}")

```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_11 (InputLayer)	(None, 20000)	0
dense (Dense)	(None, 16)	320,016
dropout (Dropout)	(None, 16)	0
dense_1 (Dense)	(None, 1)	17

Total params: 320,033 (1.22 MB)

Trainable params: 320,033 (1.22 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

625/625 ————— 13s 19ms/step - accuracy: 0.7599 - loss: 0.5057 - val_accuracy: 0.8874 - val_loss: 0.2837

Epoch 2/10

625/625 ————— 14s 9ms/step - accuracy: 0.8944 - loss: 0.2811 - val_accuracy: 0.8958 - val_loss: 0.2744

Epoch 3/10

625/625 ————— 4s 7ms/step - accuracy: 0.9164 - loss: 0.2507 - val_accuracy: 0.8924 - val_loss: 0.2915

Epoch 4/10

625/625 ————— 5s 7ms/step - accuracy: 0.9225 - loss: 0.2308 - val_accuracy: 0.8956 - val_loss: 0.2960

Epoch 5/10

625/625 ————— 5s 9ms/step - accuracy: 0.9284 - loss: 0.2256 - val_accuracy: 0.8920 - val_loss: 0.3126

Epoch 6/10

625/625 ————— 5s 7ms/step - accuracy: 0.9289 - loss: 0.2086 - val_accuracy: 0.8896 - val_loss: 0.3263

Epoch 7/10

625/625 ————— 6s 9ms/step - accuracy: 0.9349 - loss: 0.2083 - val_accuracy: 0.8876 - val_loss: 0.3365

Epoch 8/10

625/625 ————— 4s 7ms/step - accuracy: 0.9371 - loss: 0.2078 - val_accuracy: 0.8820 - val_loss: 0.3544

Epoch 9/10

625/625 ————— 5s 7ms/step - accuracy: 0.9342 - loss: 0.2132 - val_accuracy: 0.8868 - val_loss: 0.3504

Epoch 10/10

625/625 ————— 6s 8ms/step - accuracy: 0.9413 - loss: 0.1872 - val_accuracy: 0.8860 - val_loss: 0.3641

782/782 ————— 6s 8ms/step - accuracy: 0.8889 - loss: 0.2878

Test acc: 0.888

Above is no imbedding layer

```
rm -rf aclImdb
```

```
rm -rf aclImdb_v1.tar.gz
```

```
!curl -O https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
```

```
!tar -xvf aclImdb_v1.tar.gz
```

```
!rm -r aclImdb/train/unsup
```

```
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
          100 80.2M   100 80.2M    0     0  4109k      0  0:00:19  0:00:19 --:--:-- 9498k
```

```
import os, pathlib, shutil, random
from tensorflow import keras
batch_size = 32
base_dir = pathlib.Path("aclImdb")
val_dir = base_dir/"val"
train_dir = base_dir/"train"

# Create val directory if it doesn't exist
val_dir.mkdir(exist_ok=True)

for category in ("neg", "pos"):
    # Create val subdirectories with exist_ok=True
    (val_dir/category).mkdir(exist_ok=True)
    files = os.listdir(train_dir/category)
    random.Random(1337).shuffle(files)
    num_val_samples = 10000
    val_file = files[:num_val_samples]
    for fname in val_file:
        # Check if the file exists before moving
        src_path = train_dir/category/fname
        dest_path = val_dir/category/fname
        if src_path.exists() and not dest_path.exists():
            shutil.move(str(src_path), str(dest_path))
```

```

train_dir_1 = base_dir/"train1"
for category in ("neg", "pos"):
    os.makedirs(train_dir_1/category)
    files = os.listdir(train_dir_1/category)
    random.Random(1337).shuffle(files)
    num_train_samples = 100
    train_file = files[:num_train_samples]
for fname in train_file:
    shutil.move(train_dir_1/category/fname,
                train_dir_1/category, fname)

```

```

train_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/train", batch_size=batch_size
)
val_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/val", batch_size=batch_size
)
test_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/test", batch_size=batch_size
)
text_only_train_ds = train_ds.map(lambda x, y: x)

```

```

➡ Found 14900 files belonging to 2 classes.
   Found 10000 files belonging to 2 classes.
   Found 25000 files belonging to 2 classes.

```

```

from tensorflow.keras import layers

```

```

max_length = 150
max_tokens = 10000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=max_length,
)
text_vectorization.adapt(text_only_train_ds)

```

```

int_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)

```

```

inputs = keras.Input(shape=(None,), dtype="int64")
embedded = layers.Embedding(input_dim=max_tokens, output_dim=256)(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

```

```

callbacks = [
    keras.callbacks.ModelCheckpoint("embeddings_bidir_gru.keras",
                                    save_best_only=True)
]
model.fit(int_train_ds, validation_data=int_val_ds, epochs=10, callbacks=callbacks)
model = keras.models.load_model("embeddings_bidir_gru.keras")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")

```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, None)	0
embedding_1 (Embedding)	(None, None, 256)	2,560,000
bidirectional_1 (Bidirectional)	(None, 64)	73,984
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 2,634,049 (10.05 MB)

Trainable params: 2,634,049 (10.05 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

466/466 11s 20ms/step - accuracy: 0.8312 - loss: 0.4557 - val_accuracy: 0.0248 - val_loss: 1.9706

Epoch 2/10

466/466 9s 19ms/step - accuracy: 0.8559 - loss: 0.3389 - val_accuracy: 0.5709 - val_loss: 0.9595

Epoch 3/10

466/466 9s 19ms/step - accuracy: 0.8962 - loss: 0.2761 - val_accuracy: 0.4752 - val_loss: 1.0872

Epoch 4/10

466/466 9s 19ms/step - accuracy: 0.9156 - loss: 0.2299 - val_accuracy: 0.5030 - val_loss: 1.1478

Epoch 5/10

466/466 9s 19ms/step - accuracy: 0.9292 - loss: 0.1948 - val_accuracy: 0.4745 - val_loss: 1.4655

Epoch 6/10

466/466 9s 19ms/step - accuracy: 0.9398 - loss: 0.1653 - val_accuracy: 0.5443 - val_loss: 1.4006

Epoch 7/10

466/466 9s 19ms/step - accuracy: 0.9536 - loss: 0.1380 - val_accuracy: 0.5169 - val_loss: 1.6872

Epoch 8/10

466/466 9s 19ms/step - accuracy: 0.9627 - loss: 0.1093 - val_accuracy: 0.6766 - val_loss: 1.0634

Epoch 9/10

466/466 9s 19ms/step - accuracy: 0.9734 - loss: 0.0801 - val_accuracy: 0.5909 - val_loss: 1.7181

Epoch 10/10

466/466 9s 19ms/step - accuracy: 0.9789 - loss: 0.0661 - val_accuracy: 0.5409 - val_loss: 2.3702

782/782 6s 7ms/step - accuracy: 0.7460 - loss: 0.5745

Test acc: 0.744

Above is non pretrained embedded

```
rm -rf aclImdb
```

```
rm -rf aclImdb_v1.tar.gz
```

```
!curl -O https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
```

```
!tar -xf aclImdb_v1.tar.gz
```

```
!rm -r aclImdb/train/unsup
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current				
			Dload Upload	Total	Spent	Left	Speed				
100	80.2M	100	80.2M	0	0	4871k	0	0:00:16	0:00:16	--:--:--	11.8M

```
import os, pathlib, shutil, random
```

```
from tensorflow import keras
```

```
batch_size = 32
```

```
base_dir = pathlib.Path("aclImdb")
```

```
val_dir = base_dir/"val"
```

```
train_dir = base_dir/"train"
```

```
# Create val directory if it doesn't exist
```

```
val_dir.mkdir(exist_ok=True)
```

```
for category in ("neg", "pos"):
```

```
# Create val subdirectories with exist_ok=True
```

```
(val_dir/category).mkdir(exist_ok=True)
```

```
files = os.listdir(train_dir/category)
```

```
random.Random(1337).shuffle(files)
```

```
num_val_samples = 5000
```

```
val_file = files[:num_val_samples]
```

```
for fname in val_file:
```

```
# Check if the file exists before moving
```

```
src_path = train_dir/category/fname
```

```
dest_path = val_dir/category/fname
```

```
if src_path.exists() and not dest_path.exists():
```

```

shutil.move(str(src_path), str(dest_path))

train_dir_1 = base_dir/"train1"
for category in ("neg", "pos"):
    os.makedirs(train_dir_1/category)
    files = os.listdir(train_dir_1/category)
    random.Random(1337).shuffle(files)
    num_train_samples = 150
    train_file = files[:num_train_samples]
for fname in train_file:
    shutil.move(train_dir_1/category/fname,
                train_dir_1/category, fname)

```

```

train_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/train", batch_size=batch_size
)
val_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/val", batch_size=batch_size
)
test_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/test", batch_size=batch_size
)
text_only_train_ds = train_ds.map(lambda x, y: x)

```

```

➡ Found 19850 files belonging to 2 classes.
   Found 5000 files belonging to 2 classes.
   Found 25000 files belonging to 2 classes.

```

```

from tensorflow.keras import layers

```

```

max_length = 600
max_tokens = 10000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=max_length,
)
text_vectorization.adapt(text_only_train_ds)

```

```

int_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)

```

```

!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip -q glove.6B.zip

```

```

➡ --2025-04-19 14:31:58-- http://nlp.stanford.edu/data/glove.6B.zip
   Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
   Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
   HTTP request sent, awaiting response... 302 Found
   Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2025-04-19 14:31:58-- https://nlp.stanford.edu/data/glove.6B.zip
   Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
   HTTP request sent, awaiting response... 301 Moved Permanently
   Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2025-04-19 14:31:59-- https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
   Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
   Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
   HTTP request sent, awaiting response... 200 OK
   Length: 862182613 (822M) [application/zip]
   Saving to: 'glove.6B.zip'

```

```

glove.6B.zip      100%[=====] 822.24M  5.02MB/s   in 2m 39s

```

```

2025-04-19 14:34:39 (5.16 MB/s) - 'glove.6B.zip' saved [862182613/862182613]

```

```

import numpy as np
path_to_glove_file = "glove.6B.100d.txt"

```

```

embeddings_index = {}
with open(path_to_glove_file) as f:
    for line in f:
        word, coefs = line.split(maxsplit=1)
        coefs = np.fromstring(coefs, "f", sep=" ")
        embeddings_index[word] = coefs

```

```

print(f"Found {len(embeddings_index)} word vectors.")

```

```

➡ Found 400000 word vectors.

```

```

embedding_dim = 100

```

```

vocabulary = text_vectorization.get_vocabulary()
word_index = dict(zip(vocabulary, range(len(vocabulary))))

```

```

embedding_matrix = np.zeros((max_tokens, embedding_dim))
for word, i in word_index.items():
    if i < max_tokens:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector

```

```

embedding_layer = layers.Embedding(
    max_tokens,
    embedding_dim,
    embeddings_initializer=keras.initializers.Constant(embedding_matrix),
    trainable=False,
    mask_zero=True,
)

```

```

inputs = keras.Input(shape=(None,), dtype="int64")
embedded = embedding_layer(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

```

```

callbacks = [
    keras.callbacks.ModelCheckpoint("glove_embeddings_sequence_model.keras",
                                    save_best_only=True)
]
model.fit(int_train_ds, validation_data=int_val_ds, epochs=10, callbacks=callbacks)
model = keras.models.load_model("glove_embeddings_sequence_model.keras")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")

```


↔ Model: "functional_4"

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, None)	0	-
embedding_4 (Embedding)	(None, None, 100)	1,000,000	input_layer_4[0]...
not_equal_4 (NotEqual)	(None, None)	0	input_layer_4[0]...
bidirectional_4	(None, 64)	34,048	embedding_4[0][0]...